BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC CMC
KHOA CÔNG NGHỆ THÔNG TIN

BÁO CÁO TIẾN ĐỘ TRIỂN KHAI PHẦN MỀM

WorkSphere

Xây dựng nền tảng quản lý yêu cầu hỗ trợ và công việc nội bộ doanh nghiệp

Bản chia sẻ công khai

Nhóm sinh viên lớp 24IT1

Thành viên 1

Thành viên 2

Thành viên 3

Thành viên 4

Hà Nội, tháng 11 năm 2026

Báo cáo buổi trình bày ngày 11/11/2026

Bảng phân công công việc

TT	Mã SV	Họ tên	Nội dung công việc	Điểm /10	Ghi chú
1		Thành viên 1			
2		Thành viên 2			
3		Thành viên 3			
4		Thành viên 4			

Điểm tự đánh giá 30%. Giảng viên 70%.

Mã sinh viên, phần việc và điểm tự đánh giá do nhóm xác nhận trước khi nộp.

Tóm tắt buổi trình bày

Nội dung	Cập nhật của nhóm
Đã làm được gì	Đóng gói web và API bằng Docker, chạy trên Compose với PostgreSQL, thiết lập Jenkins và SonarQube, giám sát API bằng Prometheus và Grafana. Đã thử sao lưu, khôi phục và quay về image trước.
Đã làm đến đâu	Nghiệm thu trên máy cá nhân ngày 09/11/2026. Jenkins #5 đạt 407 ca backend và 8 ca web, quality gate OK, triển khai và smoke đạt. #2 và #6 xác nhận Jenkins chặn deploy khi kiểm thử hoặc quality gate lỗi.
Cần hỏi thầy	Nhóm xin xác nhận phạm vi chấm phần local, cách demo khi tài nguyên máy có hạn, và điều kiện thực hiện phần cộng điểm SSH lên máy chủ công khai. Câu hỏi cụ thể ở mục 6.4.

Demo chính: trình bày Jenkins #5, mở web Docker cổng 18210, theo dõi yêu cầu hỗ trợ từ lúc gửi tới lúc đóng, sau đó mở dashboard giám sát.

MỤC LỤC

Bảng phân công và tóm tắt buổi trình bày....1
1. Giới thiệu chung....4
2. Mô hình triển khai....5
2.1. Kiến trúc triển khai trên máy cá nhân....5
2.2. Luồng CI/CD từ GitHub đến Docker Compose....6
3. Thiết lập hệ thống CI/CD....7
4. Tự động deploy lên server qua SSH....8
5. Giám sát bằng Prometheus và Grafana....8
6. Kết luận và cải tiến....9
Kịch bản demo và câu hỏi xin ý kiến....10
Phụ lục minh chứng....11

DANH SÁCH TỪ VÀ KÝ HIỆU VIẾT TẮT

Ký hiệu	Nội dung
CI/CD	Tích hợp và triển khai liên tục
API	Giao diện lập trình ứng dụng
JDK/JRE	Bộ công cụ phát triển và môi trường chạy Java
SHA	Mã định danh commit dùng để ghim phiên bản
JVM	Máy ảo Java
p95	Ngưỡng mà 95% mẫu độ trễ nằm dưới hoặc bằng
SSH	Giao thức truy cập máy chủ từ xa

DANH SÁCH HÌNH VẼ

Hình 1. Mô hình triển khai WorkSphere trên Docker Desktop
Hình 2. Luồng kiểm thử và triển khai từ GitHub đến Docker Compose
Hình 3. Jenkins #5 triển khai qua Compose và smoke PASS
Hình 4. Jenkins #6 nhận gate ERROR và bỏ qua stage triển khai
Hình 5. SonarQube hiển thị quality gate đạt của project triển khai
Hình 6. Bản WorkSphere Docker hiển thị yêu cầu đã đóng và tệp đính kèm
Hình 7. Prometheus ghi target worksphere-api ở trạng thái UP
Hình 8. Grafana hiển thị lưu lượng, độ trễ và bộ nhớ JVM của WorkSphere

1. Giới thiệu chung

1.1. Mục tiêu học phần

Đồ án tổ chức một quy trình triển khai có thể chạy lại từ Git. Jenkins lấy đúng phiên bản, kiểm thử tự động, phân tích chất lượng và đóng gói. Sau khi triển khai, nhóm kiểm tra luồng nghiệp vụ và theo dõi tình trạng API bằng số liệu thực.

1.2. Phần mềm được chọn

WorkSphere quản lý yêu cầu hỗ trợ và công việc nội bộ. Nhân viên gửi yêu cầu, trưởng phòng điều phối, người được giao xử lý và người gửi xác nhận kết quả. Web và ứng dụng Android dùng chung API Spring Boot. Môn này triển khai web, API và PostgreSQL trên máy cá nhân.

Thành phần	Phiên bản hoặc vai trò
Backend	Java 21, Spring Boot 4.1.1, Maven Wrapper và Flyway
Web	JavaScript tĩnh, Nginx reverse proxy
Dữ liệu	PostgreSQL 15, volumes riêng cho DB, uploads và mailbox
Môi trường	Windows, Docker Desktop Linux và Node.js 24

Repo worksphere-app chứa source và Dockerfile. Repo worksphere-deployment giữ pipeline, Compose, monitoring và hướng dẫn. Commit app dùng trong đợt triển khai:

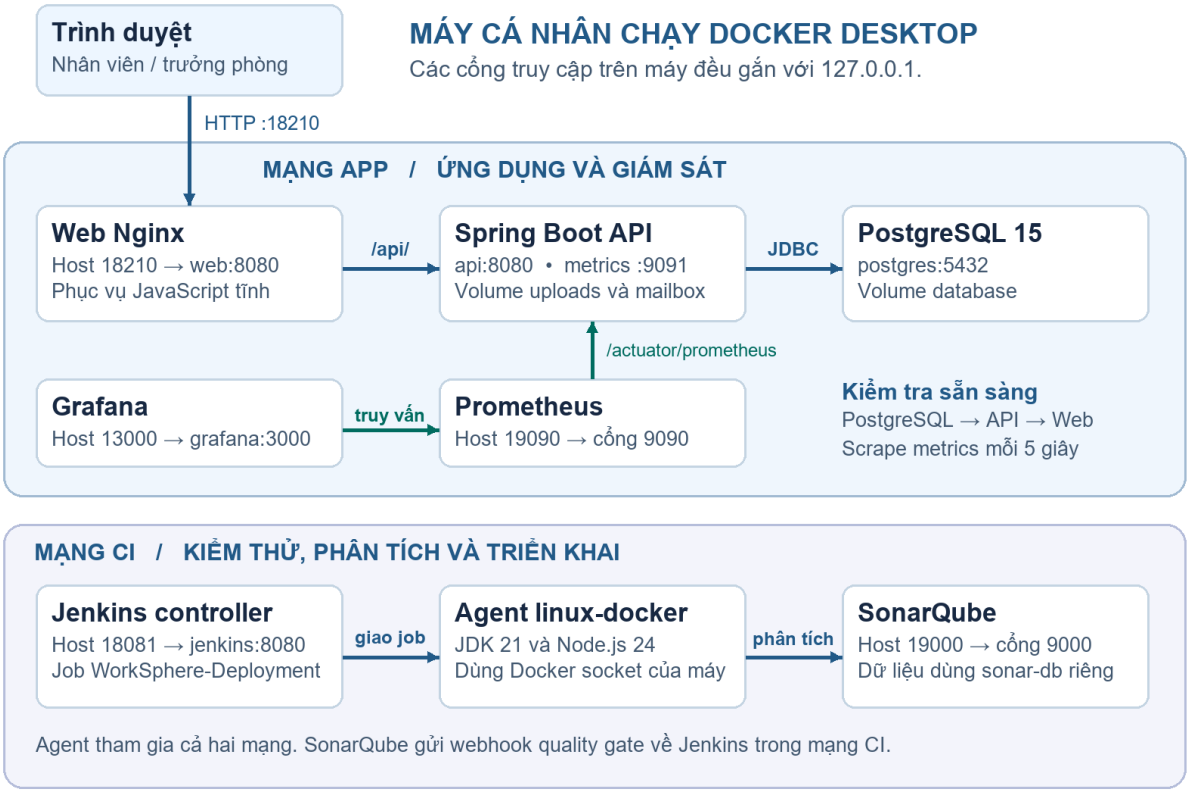
4273f1ed907eb49ae0023ab1cacff7c25de8a691

Phạm vi nghiệm thu của học phần là bản web trong Docker ở cổng 18210. Ứng dụng Android và bản web thực hành ở cổng 18110 thuộc môi trường phát triển riêng.

2. Mô hình triển khai

2.1. Kiến trúc triển khai trên máy cá nhân

Nhóm dùng ba cấu hình Compose trong cùng project worksphere-deployment. Mạng app nối web, API, PostgreSQL và giám sát. Mạng ci nối Jenkins, agent và SonarQube. Agent tham gia cả hai mạng để điều khiển Docker và kiểm thử qua web sau triển khai.



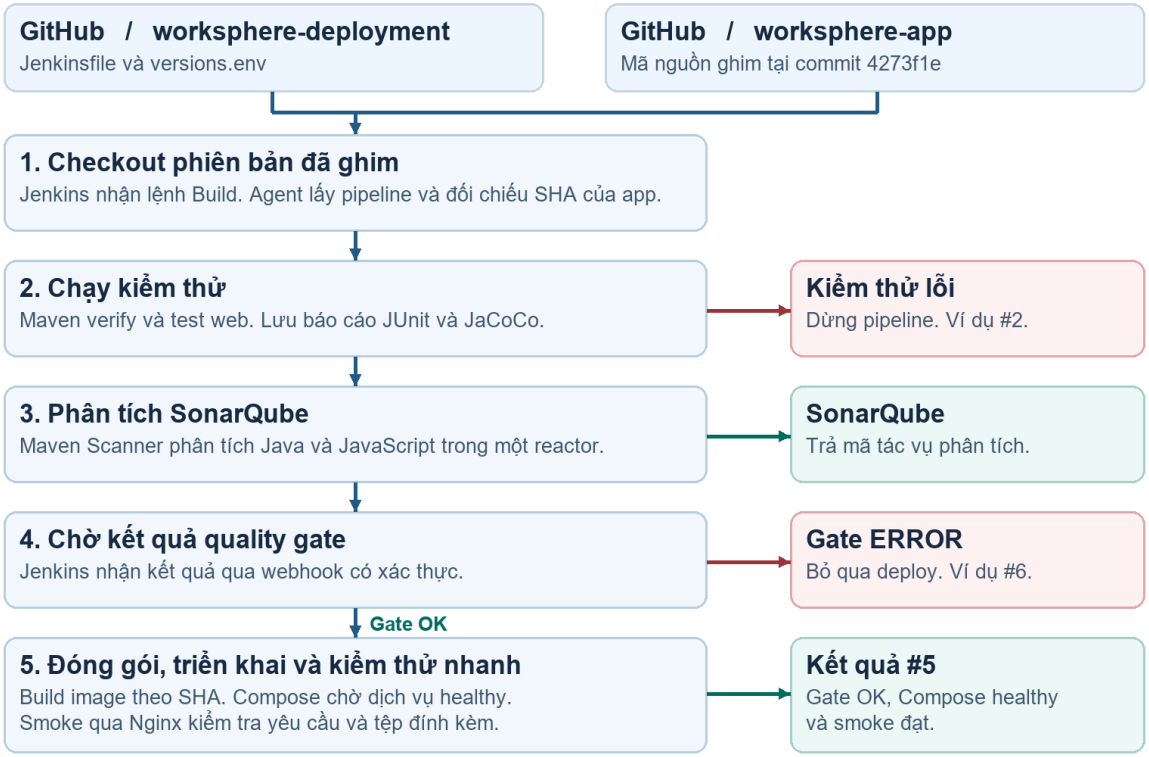
Hình 1. Mô hình triển khai WorkSphere trên Docker Desktop

Dịch vụ	Truy cập	Vai trò
Web	127.0.0.1:18210	Bản demo chạy trong Docker
Jenkins	127.0.0.1:18081	Controller và job trên máy cá nhân
SonarQube	127.0.0.1:19000	Phân tích Java, JavaScript và quality gate
Prometheus	127.0.0.1:19090	Thu metrics API theo chu kỳ 5 giây
Grafana	127.0.0.1:13000	Dashboard WorkSphere API
API và PostgreSQL	Mạng container	API 8080, metrics 9091 và PostgreSQL 5432

Nginx chuyển /api/ tới Spring Boot. Compose đợi PostgreSQL và API sẵn sàng trước khi mở web. Database, uploads và mailbox dùng volume riêng để giữ dữ liệu qua các lần tái tạo container.

2.2. Luồng CI/CD từ GitHub đến Docker Compose

Jenkins lấy Jenkinsfile từ kho triển khai. Agent đọc versions.env, checkout đúng commit của worksphere-app và đối chiếu SHA trước khi chạy test. Hình 2 thể hiện năm stage và các nhánh dừng đã kiểm chứng.



Hình 2. Luồng kiểm thử và triển khai từ GitHub đến Docker Compose

1. Tests chạy Maven verify trên JDK 21 và kiểm thử web trên Node.js 24. Testcontainers cấp PostgreSQL thử riêng cho ca tích hợp. Jenkins lưu kết quả JUnit và báo cáo độ phủ JaCoCo.

2. Maven Scanner gửi phân tích Java và JavaScript tới SonarQube. Jenkins chờ quality gate qua webhook. Lỗi test dừng các stage tiếp theo. Gate ERROR bỏ qua bước triển khai.

3. Gate OK mở stage đóng gói hai image theo SHA và chạy Compose. Khi các dịch vụ báo healthy, smoke đi qua Nginx để kiểm tra phân quyền, luồng yêu cầu hỗ trợ và tệp đính kèm. Sau khi triển khai, nhóm dùng Prometheus và Grafana để xác nhận API có số liệu.

Lượt Jenkins #5 dùng cấu hình triển khai tại commit dd20a3d. Mã image API và web đã nghiệm thu được lưu trong target.json của kho triển khai.

3. Thiết lập hệ thống CI/CD

Controller Jenkins 2.580.2 chạy trên máy cá nhân và dùng agent Linux JDK 21 với Node.js 24. Agent kiểm tra commit của app, chạy web tests, Maven, PostgreSQL thử và Testcontainers. Jenkins thu JUnit và JaCoCo XML.

Maven Scanner gửi kết quả phân tích lên SonarQube. Jenkins dùng analysis task ID và webhook có secret để nhận gate. Thời gian chờ có giới hạn. Stage Docker và Compose chỉ chạy sau khi kiểm thử và gate đạt.

Hạng mục kiểm tra	Kết quả ghi nhận
Backend P0	407 đạt, 0 thất bại, 0 lỗi, 0 bỏ qua trên JDK 21
Web P0	8/8 ca đạt trên Node.js 24
Jenkins test cố ý fail	Build #2 thất bại, stage triển khai bị bỏ qua
Jenkins đầy đủ	Jenkins #5 đạt 407 test backend, 8 test web, quality gate OK, Compose healthy và smoke đạt.
Gate từ chối	Jenkins #6 đạt 407 test nhưng SonarQube trả ERROR. Jenkins bỏ qua deploy và kết thúc FAILURE đúng kỳ vọng.
Compose và smoke	Ba dịch vụ healthy. Luồng hỗ trợ đi tới CLOSED. PNG tải lại đúng từng byte.

Ca QR trước đây trộn đồng hồ test với giờ PostgreSQL. Fixture đã dùng một mốc duy nhất cho quét hợp lệ, hết hạn và trạm đóng. Luật hết hạn của ứng dụng vẫn được kiểm tra.

Khi triển khai, Jenkins chờ các dịch vụ sẵn sàng rồi chạy kiểm thử nhanh qua Nginx. Kiểm thử gồm đăng nhập, chặn sai quyền, gửi yêu cầu kèm tệp, phân công, xử lý, đóng và tải lại tệp.

4. Tự động deploy lên server qua SSH

Mẫu xếp tự động triển khai qua SSH vào phần cộng điểm. Hiện nhóm đã nghiệm thu trên máy cá nhân và đang chờ máy chủ có quyền truy cập để thực hiện phần này.

Bước dự kiến: cấp thông tin SSH qua Jenkins Credentials, triển khai đúng image đã qua kiểm thử, chạy Docker Compose từ xa và kiểm tra URL cùng luồng nghiệp vụ. Nhóm sẽ ghi lại phiên bản image và kết quả kiểm thử trên máy chủ khi thực hiện.

5. Giám sát bằng Prometheus và Grafana

Spring Boot Actuator cung cấp số liệu qua management port 9091. Prometheus scrape trên mạng Docker. Grafana lấy số liệu từ Prometheus để vẽ trạng thái target, số request theo status, lỗi HTTP, độ trễ p95 và bộ nhớ JVM.

Sau khi Compose báo healthy, nhóm bật hai dịch vụ giám sát, kiểm tra mục Targets của Prometheus rồi mở dashboard WorkSphere API trên Grafana. Nhóm tạo lưu lượng bằng smoke test, dừng riêng API và khởi động lại để quan sát cả ba trạng thái.

Mốc	Giờ UTC+7	Trạng thái	Dữ liệu đọc qua Grafana
Trước khi dừng	11:55:27	UP	56 request, 10 mã lỗi, 56 mẫu độ trễ
API đã dừng	11:55:35	DOWN	Target tạm thời không scrape được
API phục hồi	11:55:59	UP	6 request, 2 mã lỗi, 6 mẫu độ trễ

Prometheus ghi UP → DOWN → UP. Grafana đọc metrics thật và smoke tải lại đúng tệp sau khi API phục hồi.

Các giá trị là số liệu của lượt chạy thử nhỏ. Chúng phục vụ kiểm tra khả năng giám sát và phát hiện API dừng.

6. Kết luận và cải tiến

6.1. Vận hành và phục hồi

Để sao lưu cùng thời điểm, API demo được dừng ngắn trong lúc xuất PostgreSQL và nén uploads. Manifest ghi image ID. Bản phục hồi dùng database và volumes mới trên cổng 18211. Ca smoke đã tìm được yêu cầu CLOSED và tải lại đúng tệp trong bản phục hồi.

Đã chạy lại image trước và trở về image #5 trên bản restore cổng 18211. Tệp và yêu cầu khớp ở cả hai lượt.

6.2. Kết quả và giới hạn

Đã nghiệm thu local từ checkout đến triển khai, chặn deploy khi test hoặc gate lỗi, giám sát và phục hồi. SSH public là phần cộng điểm chờ máy chủ.

Các bước tái lập và xử lý lỗi nằm trong runbooks/LOCAL.md. Môi trường kiểm tra là máy cá nhân dùng Docker Desktop Linux.

6.3. Kế hoạch hoàn thiện

Hạng mục	Việc tiếp theo
Chuẩn bị nộp	Xác nhận MSSV, phần việc và điểm tự đánh giá. Chốt thứ tự thao tác khi trình bày.
Pipeline	Giảm áp lực bộ nhớ bằng cách chạy dịch vụ theo chặng khi demo. Sau buổi trình bày, cân nhắc lưu image theo mã phiên bản trong registry để dễ đưa sang máy khác.
Vận hành	Diễn tập lại trên máy trình bày. Khi có hạ tầng, đo thời gian khôi phục và bổ sung cảnh báo API mất kết nối.
Phần mềm	Cải thiện thông báo trên web khi API tạm ngừng và kiểm thử thao tác gửi lại sau khi kết nối phục hồi.

6.4. Kịch bản demo và câu hỏi xin ý kiến

Thứ tự trình bày dự kiến

Phần	Thao tác	Kết quả cần chỉ ra
1. Phiên bản	Mở kho triển khai và Jenkins #5.	Commit đã ghim, 407 ca backend và 8 ca web đạt.
2. Điều kiện deploy	Mở quality gate và đối chiếu với Jenkins #2 và #6.	Gate OK cho phép deploy. Test lỗi hoặc gate ERROR thì deploy bị bỏ qua.
3. Web	Mở bản Docker, gửi yêu cầu kèm tệp và minh họa phân công, xử lý, xác nhận.	Yêu cầu tới CLOSED. Tệp đính kèm tải lại đúng.
4. Giám sát	Mở Prometheus Targets và dashboard Grafana.	Target UP, biểu đồ có dữ liệu. Dùng kết quả UP → DOWN → UP đã lưu để giải thích ca mất kết nối và phục hồi.

Câu hỏi nhóm xin thầy góp ý

1. Với phần bắt buộc, nhóm đã nghiệm thu đầy đủ trên máy cá nhân. Thầy muốn nhóm bổ sung minh chứng hoặc thao tác nào trước khi nộp cuối kỳ?

2. Trong thời gian trình bày, nhóm có thể mở lượt Jenkins #5 đã lưu, chạy live luồng nghiệp vụ và mở dashboard, rồi dùng #2 và #6 để minh họa ca lỗi không? Nếu cần chạy lại cả pipeline trực tiếp, nhóm sẽ chuẩn bị thời gian và tài nguyên.

3. Nếu làm phần cộng điểm SSH, nhóm có thể dùng VPS riêng hay cần môi trường của trường? Khi nghiệm thu phần này, thầy yêu cầu URL public, HTTPS hoặc điều kiện nào khác?

Phụ lục minh chứng

Ảnh chụp ngày 10/11/2026 sau khi bật lại các dịch vụ trên máy cá nhân. Jenkins #5 và #6 là các lượt chạy đã lưu ngày 09/11. Luồng smoke trên web được chạy lại ngày 10/11, xác nhận yêu cầu đã đóng và tệp đính kèm khớp.

Mốc	Kết quả
Phiên bản app	4273f1ed907eb49ae0023ab1cacff7c25de8a691
Jenkins	Jenkins #5 đạt 407 test backend, 8 test web, quality gate OK, Compose healthy và smoke đạt.
Chặn deploy	Build #2 test lỗi có chủ ý. Jenkins #6 đạt 407 test nhưng SonarQube trả ERROR. Jenkins bỏ qua deploy và kết thúc FAILURE đúng kỳ vọng.
Phục hồi	Project riêng tại cổng 18211. Yêu cầu và ảnh đính kèm khớp.
Giám sát	Prometheus ghi UP → DOWN → UP. Grafana đọc metrics thật và smoke tải lại đúng tệp sau khi API phục hồi.

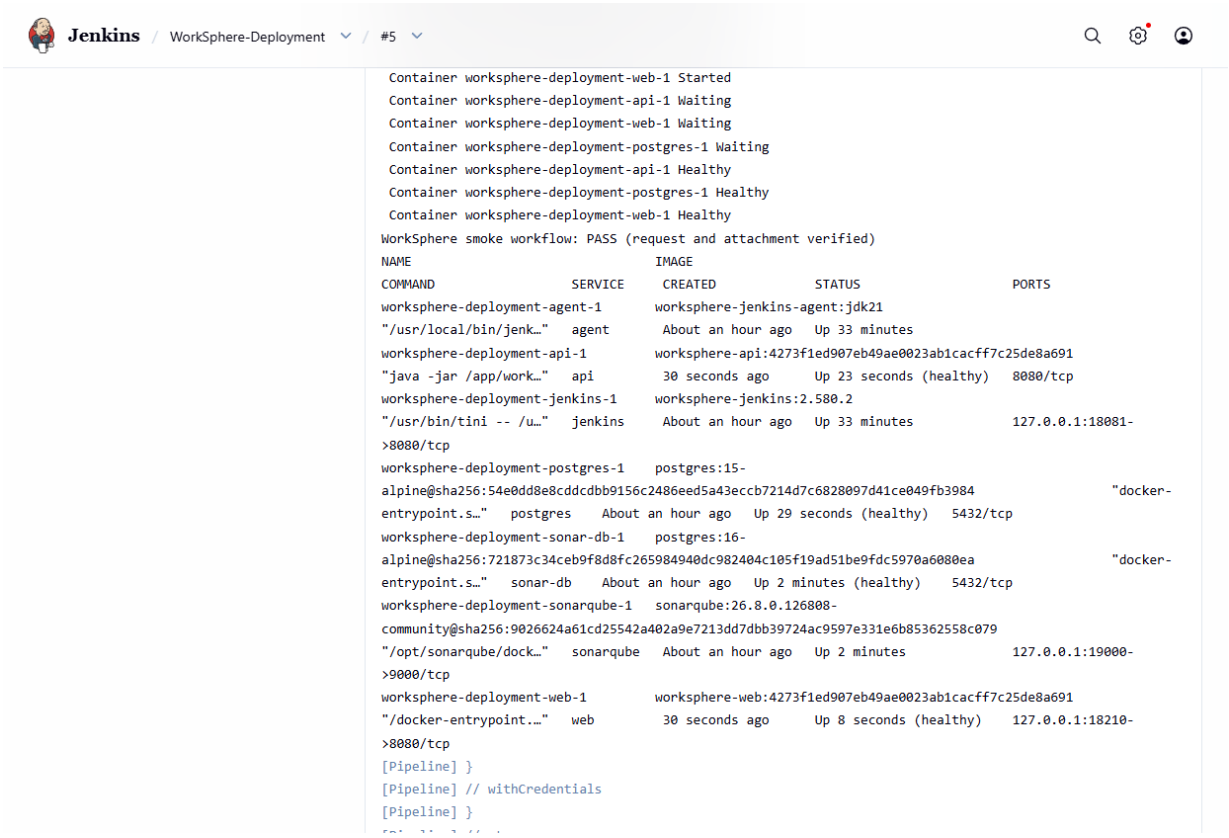
Tài liệu tham khảo

Mẫu báo cáo học phần Triển khai phần mềm, Trường Đại học CMC.

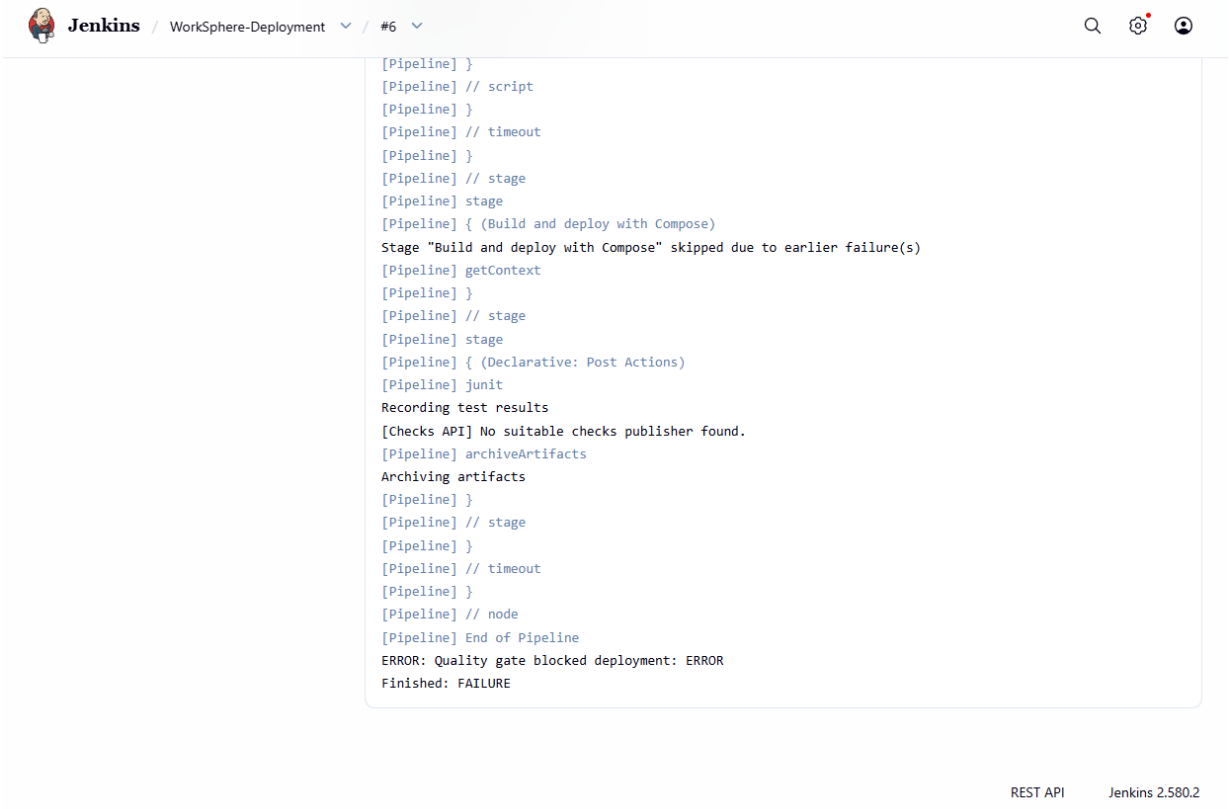
Kho triển khai — https://github.com/Khiemnh158/worksphere-deployment

Kho mã nguồn WorkSphere — https://github.com/Khiemnh158/worksphere-app

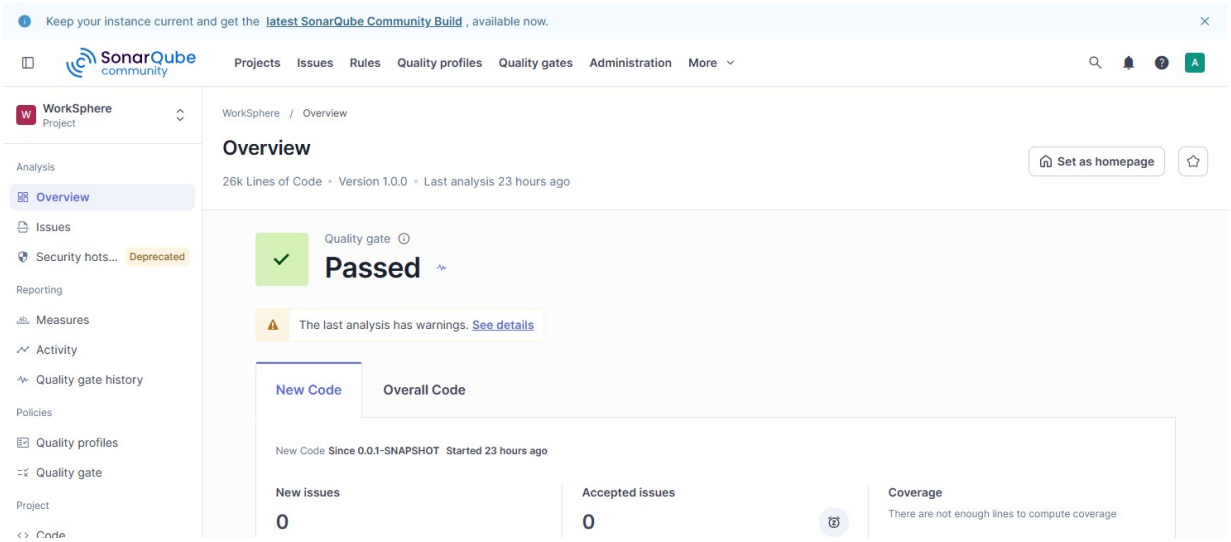
Trong kho triển khai, xem evidence/LOCAL_2026-11-09.md và runbooks/LOCAL.md.



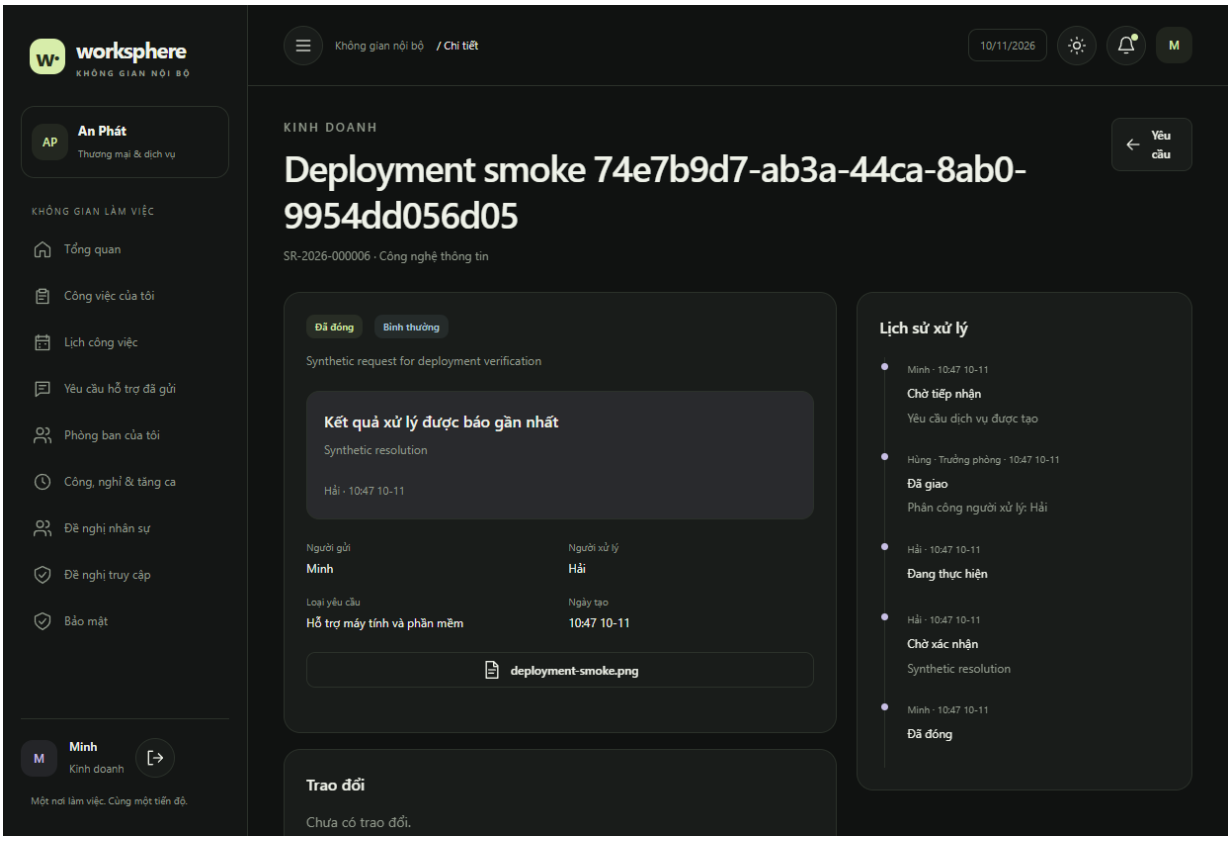
Hình 3. Jenkins #5 triển khai qua Compose và smoke PASS



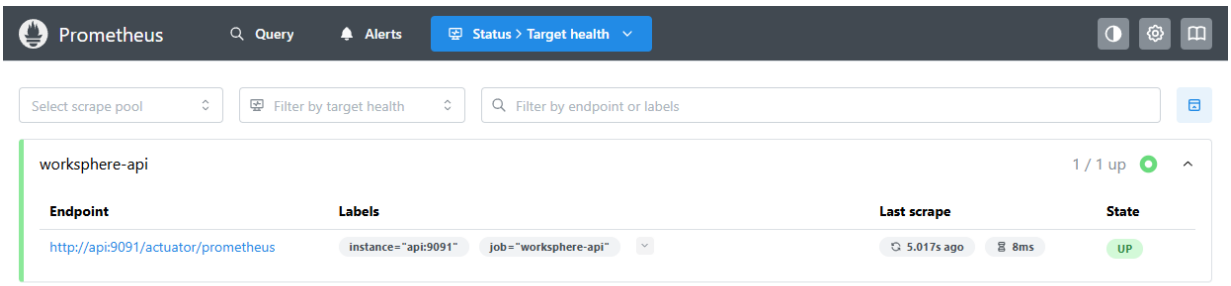
Hình 4. Jenkins #6 nhận gate ERROR và bỏ qua stage triển khai



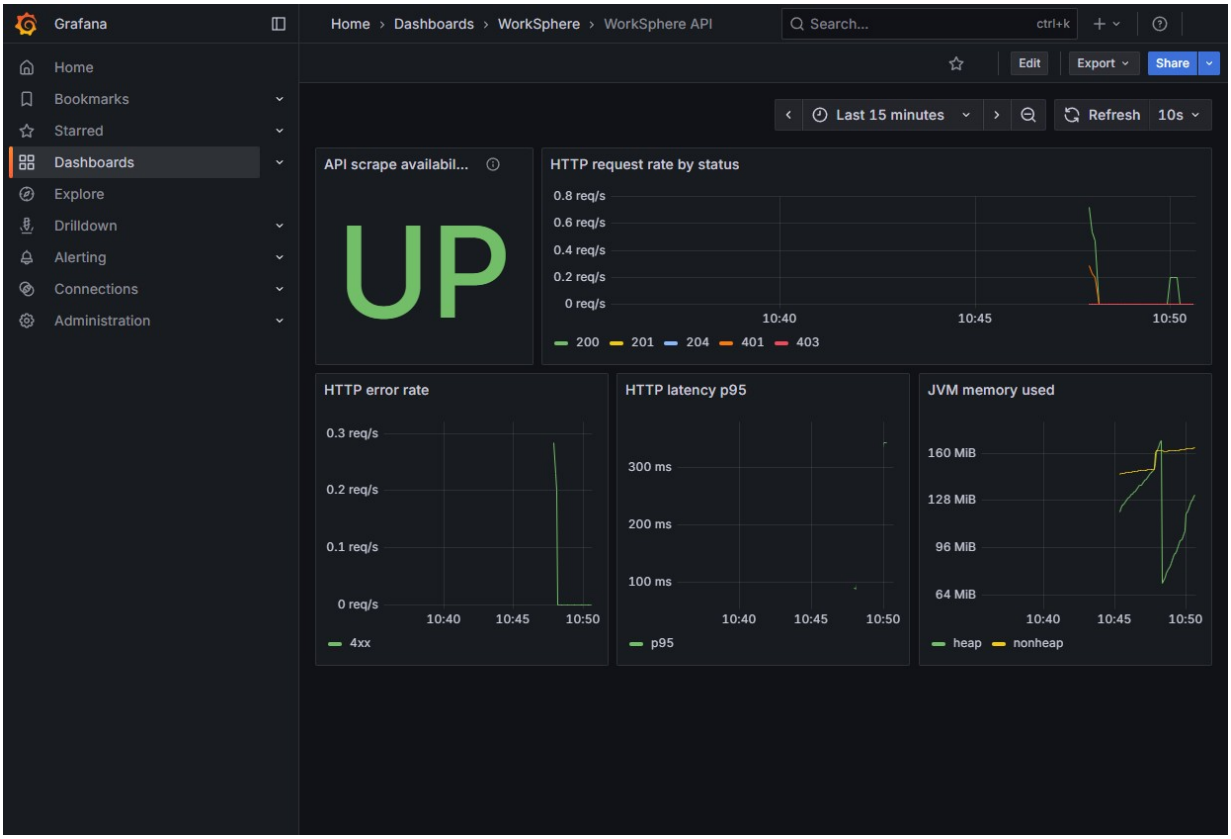
Hình 5. SonarQube hiển thị quality gate đạt của project triển khai



Hình 6. Bản WorkSphere Docker hiển thị yêu cầu đã đóng và tệp đính kèm



Hình 7. Prometheus ghi target worksphere-api ở trạng thái UP



Hình 8. Grafana hiển thị lưu lượng, độ trễ và bộ nhớ JVM của WorkSphere